🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

Confianza en almacenamiento del lado del cliente (cookie y Local Storage) para una decisión de control

ID del Ticket	**VULN-2026-005**
Activo evaluado	**Aplicación web «Inspector Challenge» (Lab Local Storage)**
Severidad	**MEDIA** **CVSS v3.1 5.3**
Tipo	**Client-Side Enforcement / Reliance on Untrusted Input — CWE-602 / CWE-565 / CWE-807**
Cliente	**Software Seguro — Laboratorio Local Storage**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**08 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 4
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 7

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

La aplicación toma una decisión de control a partir de datos que **el propio usuario puede editar libremente** desde su navegador: una cookie y un valor guardado en el "almacenamiento local" (*Local Storage*). Usando las herramientas de desarrollador que todo navegador incluye, el usuario reemplaza esos valores y la aplicación **da por superado el control**, revelando el contenido protegido.

¿Por qué importa para el negocio?

Todo lo que vive en el navegador está bajo control de la persona que lo usa: no es un lugar donde apoyar reglas de seguridad. Cuando una aplicación confía en esos datos para decidir qué mostrar o qué permitir, cualquiera puede **saltarse la restricción sin herramientas especiales**. El riesgo real depende de qué proteja ese control: si solo afecta a la propia sesión del usuario, el impacto es acotado; si el servidor también confía en esos valores para conceder permisos, el problema se vuelve grave (ver nota de escalamiento). En cualquier caso, es una **falla de diseño** que conviene corregir.

¿Qué tan grave es?

Severidad MEDIA — CVSS 5.3. La explotación es trivial y no requiere autenticación, pero el impacto demostrado se limita a la sesión del propio usuario (no afecta a terceros ni expone datos de otros). Durante la prueba se modificaron la cookie y el valor de *Local Storage* con el hash solicitado y se obtuvo el código de verificación del laboratorio.

¿Qué hay que hacer?

Mover la validación al **servidor**: ningún control de seguridad debe depender de valores almacenados en el navegador. Si un dato del cliente influye en una decisión, el servidor debe verificarlo con información que el usuario no controle.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Elusión de un control aplicado en el cliente mediante edición de cookie y Local Storage	Alta	Medio	Planificada

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

La aplicación evalúa, del lado del cliente, si una cookie y una clave de `localStorage` contienen un valor esperado (el hash MD5 de la cadena `"hackertech"`) y, de cumplirse, revela el contenido protegido. Ambos almacenes son **completamente editables por el usuario** a través del inspector del navegador o de la consola JavaScript, por lo que el control no constituye una frontera de seguridad real. Se trata de *Client-Side Enforcement of Server-Side Security*: una decisión de seguridad apoyada en entradas no confiables controladas por el cliente.

Observaciones adicionales:

- El valor esperado es un **hash MD5** (`6397550e14d36fc51b5049e28c40876f` = `md5("hackertech")`), un algoritmo obsoleto y no apto para fines de seguridad; además, al compararse en el cliente, el valor "secreto" queda expuesto en el código de la página.
- Almacenar información relevante para la seguridad en `localStorage`/cookie sin control de integridad del servidor constituye además una práctica de *almacenamiento inseguro*.

2.2 Flujo del ataque

- El usuario abre el reto y, con F12 / «Inspeccionar», accede a la pestaña de Almacenamiento (Cookies y Local Storage).
- Reemplaza el valor de la cookie indicada por el hash esperado.
- Reemplaza el valor de la clave de `localStorage` por el mismo hash.
- Recarga la página: la lógica del cliente da por superado el control y revela el código.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-602 — Client-Side Enforcement of Server-Side Security · CWE-565 — Reliance on Cookies without Validation and Integrity Checking · CWE-807 — Reliance on Untrusted Inputs in a Security Decision · CWE-922 — Insecure Storage of Sensitive Information · CWE-328 — Use of Weak Hash (MD5)
OWASP Top 10 (2021)	A04:2021 — Insecure Design · A01:2021 — Broken Access Control (control aplicado solo en cliente)
OWASP WSTG	WSTG-CLNT / WSTG-ATHZ — Testing for client-side storage y bypass de controles del lado del cliente
CVE	No aplica (defecto de diseño de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«Inspector Challenge» (Lab Local Storage)
Host	`chl-<instancia>-local-storage.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Superficies afectadas	Cookie del dominio y clave de `localStorage` evaluadas por la lógica del cliente
Dato involucrado	Valor de control (hash MD5) y contenido protegido revelado por el cliente
Alcance	Sesión del propio usuario (salvo que el servidor confíe en estos valores — ver nota de escalamiento)

2.5 Causa raíz

La lógica de autorización/validación reside en el navegador, un entorno bajo control total del usuario. Cookies y `localStorage` son almacenes de solo-cliente, legibles y modificables; no ofrecen garantía de integridad frente a

quien controla el dispositivo. Al tomar una decisión de seguridad en función de esos valores —y compararlos contra un secreto embebido en el propio código de la página— el control resulta eludible por diseño.

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1. La puntuación refleja el impacto demostrado (acotado a la propia sesión del cliente).

CVSS Base Score	Severidad	Vector
5.3	MEDIA	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	La aplicación se entrega a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: editar cookie/localStorage desde el inspector.
Privileges Required (PR)	**None (N)**	No se requiere autenticación ni privilegios previos.
User Interaction (UI)	**None (N)**	El atacante opera sobre su propio navegador; no depende de una víctima.
Scope (S)	**Unchanged (U)**	El impacto permanece dentro de la misma aplicación/autoridad de seguridad.
Confidentiality (C)	**Low (L)**	Se accede a contenido que el control pretendía restringir, limitado a la propia sesión.
Integrity (I)	**None (N)**	No se modifican datos del servidor ni de otros usuarios en el impacto demostrado.
Availability (A)	**None (N)**	No afecta la disponibilidad del servicio.

Nota de escalamiento: la puntuación 5.3 asume que el control es **puramente del lado del cliente** y solo afecta la sesión del propio usuario. Si el **servidor** confía en esa cookie o en ese valor de `localStorage` para conceder acceso o decidir estado (p. ej. tratarlos como prueba de autorización), el impacto sobre la integridad asciende a `I:H` y el vector `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N` alcanza **9.1 (Crítica)**. Se recomienda confirmar con el equipo de desarrollo si el backend valida estos valores.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier persona con un navegador puede replicar el hallazgo.

4.1 Requisitos previos

- Un navegador web con herramientas de desarrollador (F12).
- El valor esperado: `6397550e14d36fc51b5049e28c40876f`, el hash MD5 de la cadena `"hackertech"`.

4.2 Pasos de reproducción

Paso 0 — (Opcional) Calcular el hash esperado. Confirma el valor que la lógica del cliente compara.

```
echo -n "hackertech" | md5sum
# -> 6397550e14d36fc51b5049e28c40876f
```

Paso 1 — Abrir el inspector. Ingresar al reto, presionar `F12` (o clic derecho → «Inspeccionar») e ir a la pestaña **Almacenamiento** (Application/Storage).

Paso 2 — Modificar la cookie. Desplegar **Cookies**, seleccionar el dominio del sitio, hacer doble clic sobre el valor de la cookie del reto, borrar su contenido, pegar el hash y presionar Enter.

```
# Alternativa por consola (JavaScript):
document.cookie = "<nombre_cookie>=6397550e14d36fc51b5049e28c40876f; path=/";
```

Evidencia 1 — La fila de la cookie muestra el nuevo valor (el hash esperado).

Paso 3 — Modificar el valor en Local Storage. Desplegar **Almacenamiento local**, seleccionar el dominio, editar la clave del reto, borrar el contenido, pegar el hash y presionar Enter.

```
# Alternativa por consola (JavaScript):
localStorage.setItem("<clave_localStorage>", "6397550e14d36fc51b5049e28c40876f");
```

Evidencia 2 — La fila de Local Storage muestra el nuevo valor (el hash esperado).

Paso 4 — Verificación del impacto. Recargar la página: la lógica del cliente da por superado el control y entrega el código del laboratorio.

```
# Código de verificación obtenido:
#   2a67fed05c1362c6f39e7ddb95d96101
```

Evidencia 3 — El control se supera editando únicamente valores del lado del cliente, demostrando que no constituye una frontera de seguridad.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El control se valida en el servidor; editar cookie/localStorage no altera la decisión de seguridad.	Editar cookie y `localStorage` desde el inspector supera el control y revela el contenido.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado. La manipulación afectó exclusivamente al navegador del propio analista, sin impacto sobre datos de otros usuarios ni del servidor.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Trasladar la validación al servidor. Toda decisión de seguridad (qué se muestra, qué se permite) debe resolverse en el backend, usando datos que el usuario no controle (la sesión del servidor). El cliente nunca debe ser la autoridad que determina si un control se cumple.

```
# VULNERABLE (cliente): decide en el navegador comparando valores editables
if (getCookie("c") === ESPERADO && localStorage.getItem("k") === ESPERADO) {
    mostrarContenidoProtegido();          # <- el usuario controla ambos valores
}

# CORREGIDO (servidor): la decisión se toma del lado del servidor y se autoriza por sesión
@login_required
def contenido_protegido(request):
    if not request.user.has_perm("app.ver_contenido"):
        return HttpResponseForbidden()     # fuente de verdad: el servidor
    return render(request, "contenido.html")
```

5.2 Mitigaciones y buenas prácticas

- **No confiar en entradas del cliente** (cookies, `localStorage`, parámetros, cabeceras) para decisiones de seguridad; tratarlas siempre como no confiables.
- **Integridad de cookies de estado**: si una cookie transporta estado sensible, firmarla/ cifrarla del lado del servidor (p. ej. sesiones firmadas) y validar la firma en cada petición.
- **No almacenar secretos ni lógica de control en el cliente**; el código de la página es visible para el usuario.
- **Evitar MD5** para cualquier fin de seguridad; usar primitivas modernas (y, para contraseñas, algoritmos de derivación como Argon2/bcrypt).
- **Atributos de cookie** adecuados (`HttpOnly`, `Secure`, `SameSite`) cuando corresponda, como defensa en profundidad.

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Confirmar si el backend confía en estos valores (define la severidad real)	Equipo de desarrollo	Alta	Inmediato	Pendiente
Mover la validación del control al servidor	Equipo de desarrollo	Media	15 días	Pendiente
Firmar/validar cookies de estado y revisar almacenamiento en cliente	Equipo de desarrollo	Media	15–30 días	Pendiente
Reemplazar MD5 por primitivas modernas	Equipo de desarrollo	Baja	30 días	Pendiente
Re-test de verificación	Analista de seguridad	Media	Tras el fix	Pendiente

5.4 Verificación (re-test)

Confirmar que, tras editar la cookie y el valor de `localStorage` con cualquier contenido, el control **no** se supera en el cliente y que el acceso al contenido protegido se decide en el servidor en función de la sesión. Verificar además que un usuario legítimo conserva su acceso normal.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la manipulación de almacenamiento del cliente ya no altera la decisión de seguridad, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A04 Insecure Design: https://owasp.org/Top10/A04_2021-Insecure_Design/
- OWASP Top 10 (2021) — A01 Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- CWE-602 — Client-Side Enforcement of Server-Side Security: https://cwe.mitre.org/data/definitions/602.html
- CWE-565 — Reliance on Cookies without Validation and Integrity Checking: https://cwe.mitre.org/data/definitions/565.html
- CWE-807 — Reliance on Untrusted Inputs in a Security Decision: https://cwe.mitre.org/data/definitions/807.html
- CWE-922 — Insecure Storage of Sensitive Information: https://cwe.mitre.org/data/definitions/922.html
- OWASP WSTG — Testing Browser Storage: https://owasp.org/www-project-web-security-testing-guide/
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-08	J. M. F.	Creación del reporte: elusión de un control del lado del cliente mediante edición de cookie y Local Storage.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.